

Sw10 - Feature Detection

How to find a blob?

1

Image registration

Motivating example: Image registration

Goal: Align two or more images taken at different times, from different viewpoints, or by different imaging modalities, to enable comparison, analysis, or combination.

Compensate changes in:

- view angle
- distance
- orientation
- sensor resolution
- object motion

Major steps:

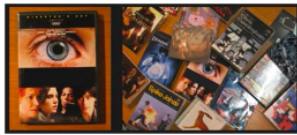
- Feature detection
- Feature matching
- Estimate geometric transformation model

Goal: Have robust features

We want to detect the same features across different images. They have to be robust against: scaling, rotation, translation, etc.

Scale-invariant feature transformation (SIFT)

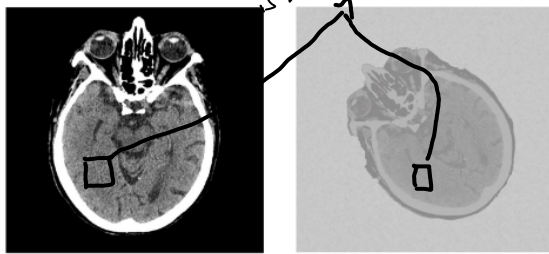
The idea of the SIFT is to find and match "interesting points or features" in images, independent of the scaling or orientation of the images and without manual intervention.



Checking the accordance of these SIFT points after completing the registration can serve as validation for the quality of our registration.

The first question we need to answer is what is an "point of interest"?
Next we will look at how to find these "points of interest" in an image.
→ SIFT detector
→ SIFT descriptor

What is an "interesting point"?

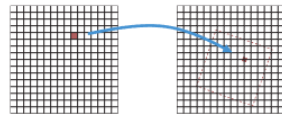


Here, interesting means that we can find a point (or region) in another image reliably!

Definition of a interesting point

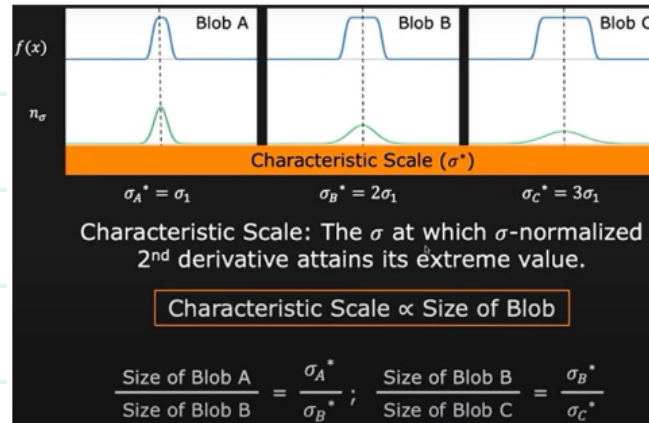
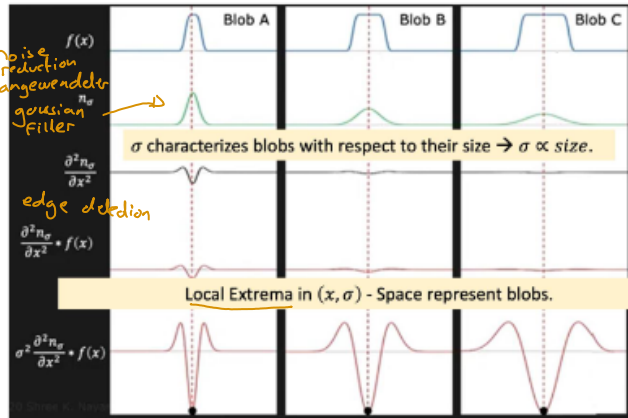
- rich information content (brightness var., color var...) in a local area
- well-defined representation
- well-defined position
- scale and rotation invariant

"Blobs" are patches with a coordinate (eine Richtung)



av

Noise reduction
angewendet
gaussian
filter



SIFT: Algorithm to find blob (1D version)

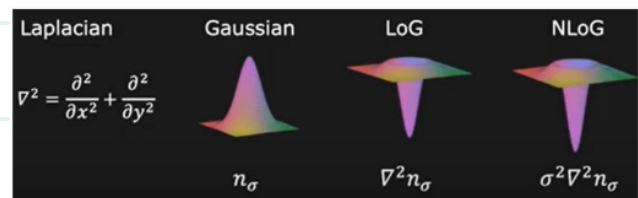
Given is a 1D signal $f(x)$

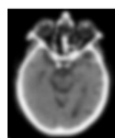
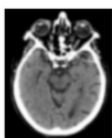
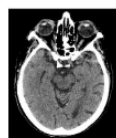
Calculate: $\sigma^2 \frac{\partial^2 n_\sigma}{\partial^2 x} * f(x)$ for many different $(\sigma_1, \sigma_2, \sigma_3, \dots, \sigma_k)$

Find: $(x^*, \sigma^*) = \arg \max_{(x, \sigma)} \left| \sigma^2 \frac{\partial^2 n_\sigma}{\partial^2 x} * f(x) \right|$
→ x^* : blob position
→ σ^* : characteristic scale

2D version for SIFT

- using Normalized Laplacian of Gaussian (NLoG)
- Location of blobs by local extrema after applying NLoG for many different scales of sigma $(\sigma_1, \sigma_2, \sigma_3, \dots, \sigma_k)$




 $S(x, y, \sigma_0)$
 $S(x, y, \sigma_1)$
 $S(x, y, \sigma_2)$

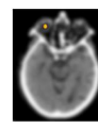
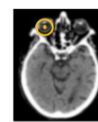
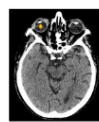
increasing σ , higher scale, lower resolution

Scale-Space: Stack created by filtering an image with Gaussians of different σ .

$$S(x, y, \sigma) = n(x, y, \sigma) * I(x, y)$$

$$\sigma^2 \nabla^2 S(x, y, \sigma)$$

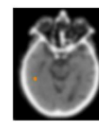
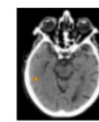
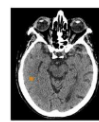
$$(Nlog * I(x, y))$$


 $S(x, y, \sigma_0)$
 $S(x, y, \sigma_1)$
 $S(x, y, \sigma_2)$

Maxima at characteristic scale σ_1

 σ_0
 σ_1
 σ_2

scale


 $S(x, y, \sigma_0)$
 $S(x, y, \sigma_1)$
 $S(x, y, \sigma_2)$

→ No maxima at any scale → no blob

 σ_0
 σ_1
 σ_2

scale

Whole algorithm: SIFT

Given is an image $I(x, y)$

One applies the NLoG for many different $(\sigma_1, \sigma_2, \sigma_3, \dots, \sigma_k)$ to create a scale-space.

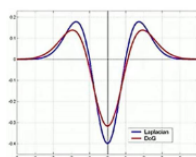
NLoG-operator

$$\text{Find: } (x^*, y^*, \sigma^*) = \arg \max_{(x, y, \sigma)} \sigma^2 \nabla^2 n_\sigma * I(x, y) \rightarrow (x^*, y^*): \text{blob position}$$

→ σ^* : blob scale
↳ Maxima of 2. and mit gaussian filter

There is one more trick involved in the practical implementation of the SIFT algorithm:
The NLoG operator can be approximated by the difference of two Gaussian functions (DoG).

$$DoG \approx (s-1)NLoG$$



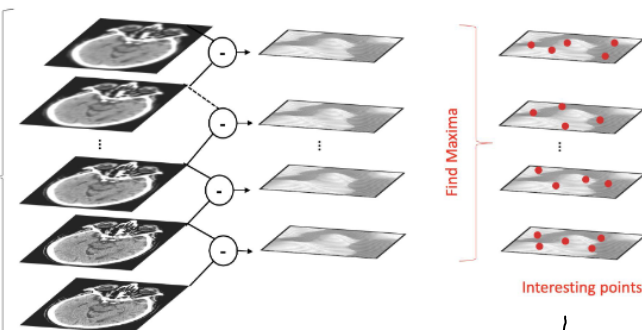
$$\text{Difference of Gaussian (DoG)} = (n_{s\sigma} - n_\sigma) \approx (s-1)\sigma^2 \nabla^2 n_\sigma$$

NLoG

illustrate



Bild $I(x, y)$

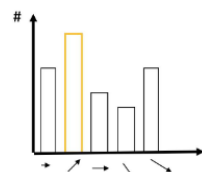
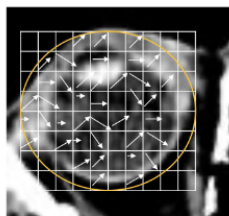


Find Maxima

Interesting points

We have with the SIFT blob two characteristics:

1. Scale invariance (knowing both scales σ_1 and σ_2)
2. Rotation invariance



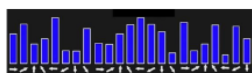
How to calculate the orientation of a SIFT blob:

1. Calculate the image gradient in every pixel: $\theta = \tan^{-1}(\frac{\partial I}{\partial x} / \frac{\partial I}{\partial y})$
2. Calculate an orientation histogram → determine the dominating direction

We know how to find SIFT blobs/points → but how to match?

⇒ We need a unique description / signature for each SIFT blob/point

Using directional histograms for different areas of the SIFT blob, one can create a signature of the SIFT blob.



These signatures can then be compared with one another using various metrics:

L2 Distance:

$$d(H_1, H_2) = \sqrt{\sum_k (H_1(k) - H_2(k))^2}$$

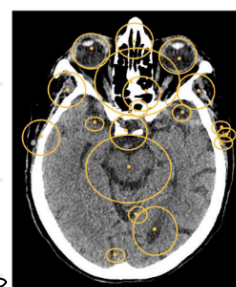
→ Perfect match for $d(H_1, H_2)=0$

Normalized Correlation:

$$d(H_1, H_2) = \frac{\sum_k (H_1(k) - \bar{H}_1)(H_2(k) - \bar{H}_2)}{\sqrt{\sum_k (H_1(k) - \bar{H}_1)^2} \sqrt{\sum_k (H_2(k) - \bar{H}_2)^2}}$$

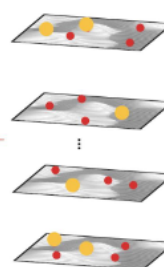
→ Perfect match for $d(H_1, H_2)=1$

$$\text{where: } \bar{H}_i = \frac{1}{N} \sum_{k=1}^N H_i(k)$$



SIFT interest points aren't classical blobs and often not visually intuitive.

Find Maxima



Interesting points

strong Maxima

→ SIFT interest points

SIFT — Scale-Invariant Feature Transform

Was ist SIFT überhaupt?

SIFT ist ein Algorithmus um **markante Punkte (Keypoints)** in Bildern zu finden und beschreiben — so dass man sie **wiedererkennt**, egal ob das Bild:

- größer/kleiner ist (Scale)
- rotiert ist (Rotation)
- heller/dunkler ist (Belichtung)

Die 4 Schritte

Schritt 1: Scale Space — Gauss-Pyramide aufbauen

Man filtert das Bild mit **immer stärkerem Gauss** → mehrere "Unschärfe-Stufen":

Original → Blur σ_1 → Blur σ_2 → Blur σ_3 → ...

Dann bildet man die **Differenz** zwischen benachbarten Stufen:

$$DoG = G(x, y, \sigma_2) - G(x, y, \sigma_1)$$

DoG = **Difference of Gaussians** — das ist eine Annäherung an den Laplacian (LoG)!

Gauss-Pyramide:	DoG-Pyramide:
[Blur 1]	[Blur2 - Blur1]
[Blur 2]	[Blur3 - Blur2]
[Blur 3]	[Blur4 - Blur3]
[Blur 4]	...

Matching: Wie vergleicht man zwei Bilder?

Bild A Keypoints	Bild B Keypoints
[128-dim Vektor]	→ finde nächsten Nachbar im 128-dim Raum

Zwei Keypoints matchen wenn ihr **euklidischer Abstand** klein ist:

$$d = \sqrt{\sum_{i=1}^{128} (a_i - b_i)^2}$$

Warum ist SIFT invariant?

Invarianz	Wie gelöst
Scale	DoG-Pyramide — Keypoints auf mehreren Skalen gesucht
Rotation	Descriptor relativ zur dominanten Orientierung
Helligkeit	Gradienten statt absolute Pixelwerte
Translation	lokale Deskriptoren, nicht global

🔑 Kern-Intuition

Schritt 1: DoG-Pyramide	→ "auf welcher Skala ist der Punkt interessant?"
Schritt 2: Extrema finden	→ "welche Punkte sind wirklich markant?"
Schritt 3: Orientierung	→ "in welche Richtung schaut der Punkt?"
Schritt 4: Descriptor	→ "wie sieht die Umgebung aus?" (128D Fingerabdruck)

SIFT ist so robust weil er **nicht absolute Werte** vergleicht, sondern immer **relative Strukturen** — genau wie ein Mensch ein Gesicht erkennt, egal ob es beleuchtet, gedreht oder weiter weg ist.

Schritt 2: Keypoint-Detektion — Extrema finden

In der DoG-Pyramide sucht man **lokale Extrema** (Maxima & Minima):

Ein Punkt ist ein Keypoint wenn er **größer oder kleiner** ist als alle 26 Nachbarn:

Aktueller Pixel → vergleiche mit:
8 Nachbarn gleiche Ebene
9 Nachbarn Ebene drüber
9 Nachbarn Ebene drunter
= 26 Vergleiche total

→ Nur wirklich **markante Punkte** überleben (Ecken, Blob-Strukturen)

Schritt 3: Keypoint-Orientierung

Damit SIFT **rotationsinvariant** wird, berechnet man für jeden Keypoint:

- Gradienten-Richtung aller Nachbarpixel
- Histogramm der Richtungen (36 Bins = je 10°)
- Dominante Richtung** = Orientierung des Keypoints

Histogramm:



→ Später wird alles **relativ zu dieser Richtung** beschrieben

Schritt 4: Feature Descriptor — der Fingerabdruck

Jetzt wird der eigentliche **128-dimensionale Descriptor** berechnet:

16×16 Fenster um den Keypoint
↓
aufgeteilt in 4×4 = 16 Subregionen
↓
jede Subregion → Histogramm mit 8 Richtungen
↓
16 × 8 = 128 Werte = der Descriptor

Dieser Vektor ist der **"Fingerabdruck"** des Keypoints.

